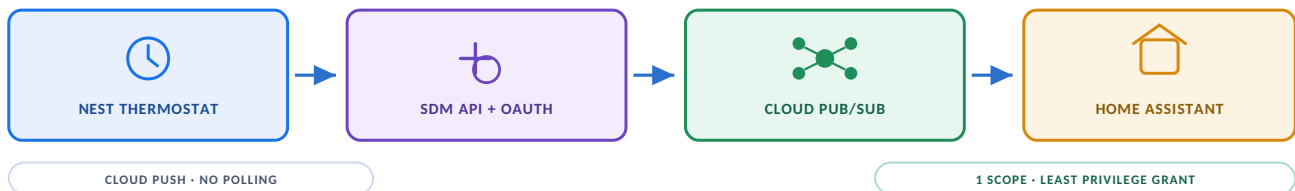


Google Cloud OAuth & Nest Device Smart Home Integration

An identity-driven, event-based integration that connects a Google Nest Thermostat to a locally hosted Home Assistant control plane through Google Cloud — using OAuth 2.0, the Smart Device Management API, and Cloud Pub/Sub. No vendor subscription, no polling, and no device credentials ever stored outside Google's identity boundary.

EVENT & CONTROL PATH

device telemetry → google-managed backend → push messaging → local control plane



Esmeralda Cabrera Ventura

Solutions Architect & Founder, Apex Intelligent Systems

U.S. Air Force Veteran · M.S. Computer Science · Cloud · AI/ML · Automation

Every screenshot in this report is an authentic capture from the live build. Project numbers, project IDs, OAuth client IDs, client secrets, account identifiers and structure identifiers have been permanently redacted.

PLATFORMS & SERVICES USED

Google Cloud Platform

Smart Device Management API

Google Auth Platform (OAuth 2.0)

Cloud Pub/Sub

Device Access Console

Cloud Monitoring

Cloud Logging

Home Assistant

Google Nest Thermostat

My Home Assistant Redirect

Service names above are referenced for descriptive purposes only and are trademarks of their respective owners.

Table of Contents

1. Executive Summary	4
2. Problem, Objectives & Significance	5
2.1 The Problem	5
2.2 Objectives	5
2.3 Why It Matters	5
3. System Architecture	6
3.1 Design Principles	6
3.2 Component Responsibilities	6
3.3 The Trust Chain	6
3.4 Two Planes: Control and Telemetry	7
3.5 End-to-End Architecture	8
4. Technology Stack	9
5. Identity & Access Design — Technical Deep Dive	10
5.1 Why the Authorization Code Flow	10
5.2 The Three-Project Identity Model	10
5.3 Scope Design and Least Privilege	11
5.4 The Redirect URI Constraint	11
5.5 Publishing Status and Token Longevity	12
5.6 Token Lifecycle	12
5.7 Client Secret Rotation	13
6. Implementation Walkthrough	14
6.1 The Cloud Project as Billing and API Boundary	14
6.2 Enabling the Smart Device Management API	15
6.3 The Device Access Project	15
6.4 OAuth Application State and Audience	17
6.5 The OAuth Client and Redirect Binding	17
6.6 Verifying the Authorization Grant	18
6.7 Confirming Live API Traffic	19
6.8 The Pub/Sub Event Pipeline	20
6.9 End-to-End Validation on the Live System	22
7. Failure Analysis & Troubleshooting	23
7.1 The Silent Authentication Failure	23
7.2 Failure Taxonomy	23
7.3 Time-Delayed Failure Modes	24
8. Security Posture	25

8.1 What the Design Protects	25
8.2 Threat Model	25
8.3 Residual Risk — An Honest Assessment	25
9. Cost Analysis	27
10. Observability & Operations	28
11. Next Phase: The AI Agent Control Layer	29
11.1 The Architectural Boundary	29
11.2 What the Agent Can Observe	29
11.3 Capability Model	30
11.4 Proposed Architecture	30
11.5 Guardrails and Safety Design	31
12. Outcomes & Skills Demonstrated	32
Appendix A — Reference Tables	33
Appendix B — Glossary	34
Appendix C — Reproduction Checklist	35
Appendix D — Figure Index	36

1. Executive Summary

This project connects a Google Nest Thermostat to a self-hosted Home Assistant instance using Google's official Smart Device Management (SDM) platform. It is not a device pairing exercise. Nest hardware exposes no local API, so the only sanctioned path runs through Google's cloud — which turns the task into an **identity and integration engineering problem**: registering a first-party OAuth relying party, constructing a four-link trust chain across two separate Google consoles, and standing up an event-driven messaging pipeline that pushes device state to a machine sitting behind a residential NAT.

The delivered system is a cloud-to-local hybrid. Google Cloud acts as a secure broker: it holds the identity boundary via OAuth 2.0, exposes device state and control through the SDM REST API, and pushes real-time change events through Cloud Pub/Sub. Home Assistant is the local control plane — it stores the refresh token, consumes events as they arrive, normalizes them into entities, and issues authenticated commands back along the same pipeline. The Nest device itself never talks to the local network at all.

1

OAUTH SCOPE GRANTED

0

POLLING LOOPS

\$0

RECURRING MONTHLY COST

9

VERIFIED CHECKPOINTS

Three design decisions define the result. First, **events over polling**: Cloud Pub/Sub delivers thermostat state changes as they happen, so the dashboard reflects the physical device in near real time without a single scheduled request. Second, **least privilege**: the integration requests the narrow `sdm.service` scope rather than broad account access, and device authorization is deliberately kept in a separate console from cloud resource management. Third, **local control plane**: automations, history and state all live on hardware the operator owns, so the cloud is a broker rather than a dependency for day-to-day logic.

The build also produced a genuinely instructive failure. The OAuth flow initially completed on Google's side and then died on return with no useful error — the classic redirect-boundary mismatch that occurs when a publicly registered callback URL cannot resolve to a service living on a private address. Section 7 documents the root cause and the fix in full, because the reasoning generalizes to every OAuth integration involving a non-public relying party.

The next phase, specified in Section 11, adds an AI agent above Home Assistant as an intelligent orchestration layer. The agent is deliberately given no cloud credentials and no direct API access. It reasons over the normalized state, event stream and history that Home Assistant already exposes, and issues intent-based actions back through the existing authenticated pipeline — which keeps the credential blast radius unchanged while adding adaptive control.

2. Problem, Objectives & Significance

2.1 The Problem

A Google Nest Thermostat is a closed appliance. It has no local HTTP API, exposes no LAN control protocol, and will not respond to anything on the home network. It maintains one outbound relationship: to Google's backend. Any platform that wants to read its temperature or change its setpoint must go through Google, as an authenticated third party, on Google's terms.

Those terms have narrowed over time. The original "Works with Nest" partner program was retired, leaving the Smart Device Management API as the only supported route to Nest hardware. Access is not granted by pressing a button on the device. It requires enrolling in a developer program, standing up a Google Cloud project, registering an OAuth client, creating a second project in a separate Device Access Console, and linking the two so a homeowner's consent grant can be exchanged for tokens scoped to their devices.

This creates the real difficulty. Home Assistant runs on a machine at a private address behind a residential router, and Google's identity platform will not accept a callback it cannot publicly resolve and verify over TLS. The relying party and the identity provider therefore sit on opposite sides of a boundary that OAuth expects to be continuous. Reconciling that gap, rather than any device configuration, is the substance of the work.

THE CORE QUESTION

How do you complete a standards-compliant OAuth 2.0 authorization code flow, and then sustain a real-time event stream, between a public cloud identity provider and a private, non-routable service — without exposing that service to the internet and without paying for a hosted relay?

2.2 Objectives

- Establish a dedicated Google Cloud project to serve as the explicit security, billing and API boundary for the integration.
- Enable the Smart Device Management API as the sole authoritative interface to Nest device telemetry, state and control.
- Design an OAuth 2.0 identity layer that grants the minimum scope necessary and survives long-lived unattended operation.
- Resolve the redirect-boundary conflict between a public identity provider and a privately hosted relying party.
- Establish device trust in a separate Device Access project, distinct from cloud resource management.
- Replace polling with an event-driven Cloud Pub/Sub pipeline delivering device changes as they occur.
- Keep the entire solution inside free-tier consumption, with no recurring subscription.
- Verify every layer independently — API enablement, consent grant, live traffic, message pipeline and rendered device state.
- Leave a clean architectural seam for an AI orchestration layer that never touches cloud credentials.

2.3 Why It Matters

The surface area exercised here is disproportionate to the size of the deliverable. A single thermostat integration required a full OAuth 2.0 authorization code flow with an external consent audience, cross-console trust delegation between two Google project types, scope minimization, publish-state management, asynchronous message-queue plumbing, credential rotation and production observability — all validated against a live system rather than a mock.

It is also a faithful model of a pattern that recurs constantly in enterprise cloud work: a third-party platform holds the data, a private internal system needs it, and the two must be joined through federated identity and asynchronous messaging without opening the network perimeter. The thermostat is incidental — the pattern is the point, as is the discipline of diagnosing the silent failure in Section 7 by reasoning about identity boundaries rather than by trial and error.

3. System Architecture

3.1 Design Principles

Four principles governed every decision in the build, and each one is traceable to a specific configuration choice documented later in this report.

PRINCIPLE	WHAT IT MEANS HERE	HOW IT WAS ENFORCED
Cloud as broker, not as dependency	Google mediates identity and transport, but day-to-day automation logic lives locally.	Home Assistant holds state, history and automations; the cloud path carries only auth and events.
Least privilege	The grant should permit device interaction and nothing else.	A single SDM service scope; device trust isolated in a separate Device Access project.
Events over polling	State should be pushed on change, not fetched on a timer.	Cloud Pub/Sub topic and subscription carry device change events to the local consumer.
Explicit boundaries	Billing, API surface, identity and device trust should each have a named owner.	One dedicated Cloud project as the API/billing boundary; a separate console for device trust.

3.2 Component Responsibilities

Six components participate. Assigning each a single, non-overlapping responsibility is what keeps the trust chain auditable.

COMPONENT	OWNER	RESPONSIBILITY
Nest Thermostat	Google	Produces telemetry and accepts commands. Communicates exclusively with Google's backend; never addressable from the local network.
Nest / SDM Backend	Google	Authoritative device state store. Exposes the REST surface for queries and commands, and emits change events.
Google Cloud Project	Operator	The security, billing and API boundary. Hosts API enablement, the OAuth client, and the Pub/Sub resources.
Device Access Project	Operator	The Nest-specific access control layer. Binds an OAuth client to the SDM scope and defines which device traits are reachable.
Cloud Pub/Sub	Operator	Asynchronous transport. Decouples event production from consumption and buffers messages when the consumer is offline.
Home Assistant	Operator	Local control plane. Stores the refresh token, consumes events, normalizes them into entities, and issues authenticated commands.

3.3 The Trust Chain

Authorization is not a single grant. It is a chain of four links, and a break at any link produces a failure that surfaces somewhere else entirely — which is precisely what makes this class of integration hard to debug.

LINK	RELATIONSHIP	ESTABLISHED BY
1	Cloud project → SDM API	Enabling <code>smartdevicemanagement.googleapis.com</code> on a billing-enabled project. Without this, tokens are valid but every API call fails.
2	Cloud project → OAuth client	Creating a web-application client with an exactly matching authorized redirect URI, under an external, in-production consent audience.

LINK	RELATIONSHIP	ESTABLISHED BY
3	Device Access project → OAuth client	Registering the client ID inside the Device Access Console so Google knows which application is permitted to request device scopes.
4	Homeowner → application	An interactive consent grant recorded as a partner connection against the home structure, exchanged for a refresh token held by Home Assistant.

WHY THIS IS THE HARD PART

Links 1 through 3 are configured in three different consoles, and none of them validates the others. A missing API enablement, a mismatched redirect URI and an unregistered client ID all produce different symptoms at different stages — one at request time, one at callback, one at token exchange. Section 7 maps each symptom back to its originating link.

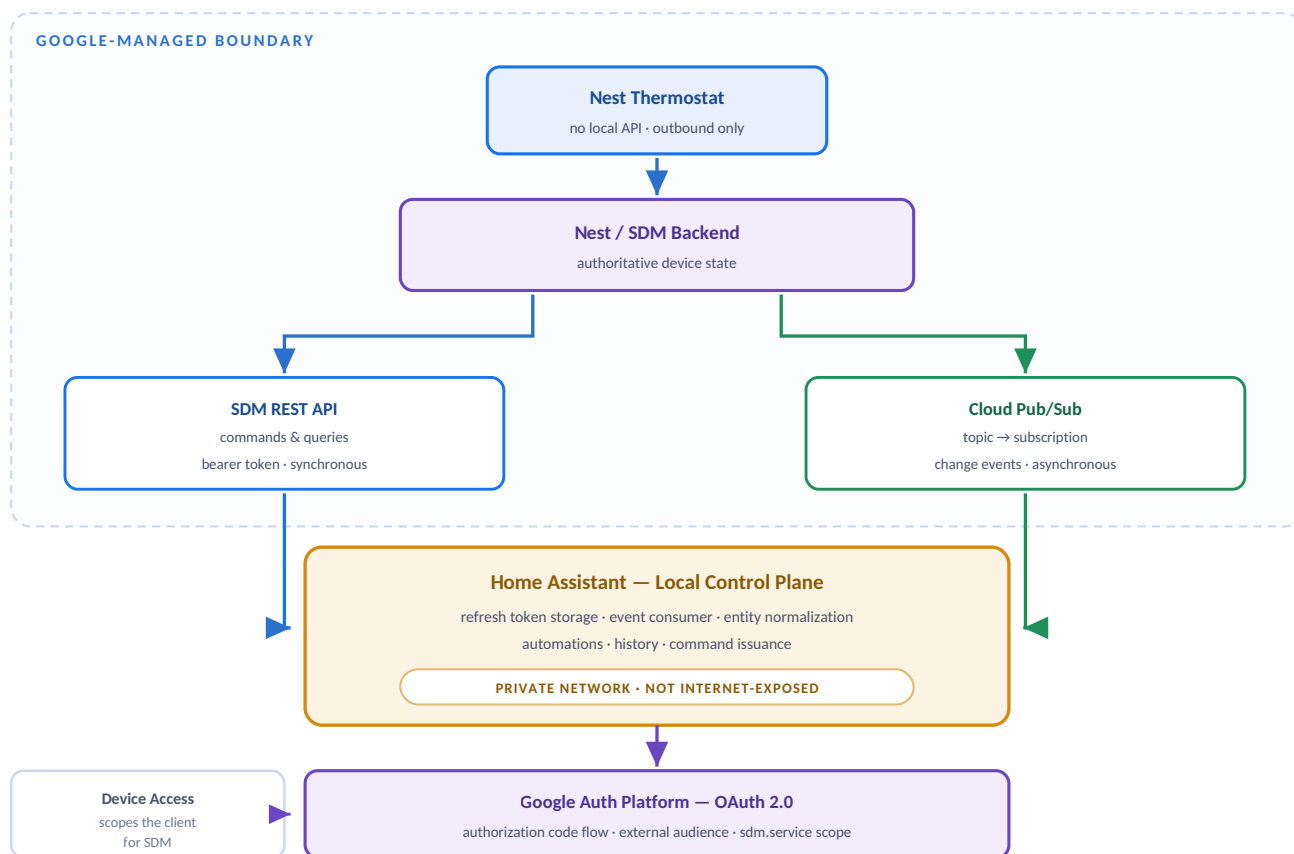
3.4 Two Planes: Control and Telemetry

The architecture carries two logically distinct flows over the same trust relationship, and separating them clarifies both the performance characteristics and the failure modes.

PLANE	DIRECTION	TRANSPORT	CHARACTERISTICS
Control plane	Home Assistant → Google	SDM REST API over HT-TPS, bearer token	Synchronous, request/response. Used for setpoint changes, mode changes and initial state hydration. Latency is user-visible.
Telemetry plane	Google → Home Assistant	Cloud Pub/Sub pull subscription	Asynchronous, push-driven. Carries device change events. Buffers during local downtime; no request is ever issued by the consumer on a timer.

This split is why Home Assistant classifies the integration as *Cloud Push* rather than *Cloud Polling*. The practical consequence is that adjusting the thermostat physically at the wall propagates to the dashboard within seconds, and it costs zero requests to keep it that way while nothing is changing.

3.5 End-to-End Architecture



READING THE DIAGRAM

The dashed boundary at the top is entirely Google-operated — the operator configures it but does not run it. Everything below the amber box is on hardware the operator owns. The two arrows crossing between them are the only paths in the system, and both are authenticated by a single OAuth grant issued once and refreshed thereafter.

4. Technology Stack

TECHNOLOGY	LAYER	ROLE IN THIS BUILD
Google Cloud Platform	Foundation	Dedicated project acting as the security, billing and API boundary. Every resource in the integration is scoped to it.
Smart Device Management API	Device interface	<code>smartdevicemanagement.googleapis.com</code> — the authoritative and only supported interface for third-party access to Nest telemetry, state and control.
Google Auth Platform	Identity	OAuth 2.0 authorization code flow. External user type, promoted to in-production publishing status. Issues and refreshes the tokens Home Assistant holds.
OAuth 2.0 Web Application Client	Identity	Represents Home Assistant as a registered relying party, with an exactly matching authorized redirect URI and a rotatable client secret.
Device Access Console	Device trust	A separate project type that binds the OAuth client to the SDM scope and enumerates which device traits are reachable. Deliberately distinct from cloud resource management.
Cloud Pub/Sub	Messaging	Topic and subscription carrying Nest device change events. Provides asynchronous decoupling, ordered delivery and buffering during consumer downtime.
Cloud Monitoring & Logging	Observability	Confirms API request volume, surfaces error conditions, and provides dashboards for the Pub/Sub pipeline.
Home Assistant	Control plane	Self-hosted automation platform. Holds credentials, consumes events, normalizes device data into entities, and issues authenticated commands.
My Home Assistant redirect	Identity bridge	A publicly resolvable TLS endpoint that satisfies Google's redirect validation rules and forwards the authorization response to the private local instance.
Google Nest Thermostat	Device	The physical endpoint. Exposed to Home Assistant as a climate entity plus discrete temperature and humidity sensor entities.

NOTABLE FOR WHAT IS ABSENT

There is no compute in this architecture — no VM, no container, no serverless function. No database, no reverse proxy, no tunnel service, and no inbound firewall rule. The integration is assembled entirely from managed identity and messaging primitives plus one piece of local software. That minimalism is a design outcome, not a limitation: every component removed is a component that cannot fail, cost money, or require patching.

5. Identity & Access Design — Technical Deep Dive

Identity is the substance of this project. The messaging layer is a few clicks; the device is a consumer appliance. What takes engineering judgment is constructing an authorization model that is correct, minimal, and durable enough to run unattended for months. This section explains the reasoning behind each choice before Section 6 shows the resulting configuration.

5.1 Why the Authorization Code Flow

OAuth 2.0 defines several grant types, and only one is appropriate here. The integration must act on behalf of a specific homeowner, against their specific devices, over a long period, without a human present to re-authenticate. That set of requirements eliminates the alternatives.

GRANT TYPE	VERDICT	REASONING
Client credentials	Rejected	Authenticates an application, not a user. It cannot express "this homeowner consents to this app reaching these devices," which is the entire authorization question.
Implicit	Rejected	Returns the token directly through the browser and issues no refresh token. Deprecated for good reason, and unusable for unattended long-lived access.
Device authorization	Rejected	Designed for input-constrained clients. Home Assistant has a full browser-based UI available during setup, so the added indirection buys nothing.
Authorization code	Selected	User-delegated, exchanges a short-lived code for tokens over a back channel, and — critically — issues a refresh token that sustains access without further interaction.

The refresh token is the operationally significant artifact. The access token expires within the hour; the refresh token is what allows the integration to keep working at three in the morning six weeks later. Everything in Section 5.5 about publishing status exists to protect it.

5.2 The Three-Project Identity Model

A frequent point of confusion in SDM integrations is that three distinct identities are involved, living in two different consoles, each with a different lifecycle. Conflating them is the most common cause of an integration that authenticates successfully and then returns no devices.

IDENTITY	WHERE IT LIVES	WHAT IT GOVERNS
Cloud project	Google Cloud Console	Billing account attachment, API enablement, Pub/Sub resources, and quota. Answers: <i>which services may be called at all?</i>
OAuth client	Google Auth Platform (within the Cloud project)	Application registration, redirect URIs, client secret, consent audience. Answers: <i>which application is asking, and where may the answer be sent?</i>
Device Access project	Device Access Console (separate)	Binding of the OAuth client to the SDM scope, and the set of device traits made reachable. Answers: <i>which devices may this application see?</i>

The separation is deliberate rather than incidental. Cloud resource management and device authorization have different blast radii and different audiences, so keeping them in separate projects means a change to one cannot silently widen the other. It also means the Device Access project can be revoked outright — severing all device visibility — without disturbing billing, API enablement or any other resource in the Cloud project.

5.3 Scope Design and Least Privilege

The integration requests a deliberately narrow permission set. The primary scope is `https://www.googleapis.com/auth/sdm.service`, which authorizes interaction with enrolled Nest devices and nothing else. It conveys no access to Gmail, Drive, contacts, location history or profile data, and it cannot be widened after the fact without a fresh consent prompt that the homeowner would see.

SCOPE	PURPOSE	BOUNDARY
<code>.../auth/sdm.service</code>	Device access	Read state and issue commands for devices explicitly enrolled in the Device Access project. No account data.
<code>.../auth/pubsub</code>	Event transport	Requested by the setup flow so the subscription carrying device events can be created and consumed on the operator's own project.

A second layer of restriction sits underneath the scope. The Device Access project enumerates the device traits it exposes — Structure, Room, Thermostat, Camera, Doorbell and Display. Even with a valid token, the application can only reach trait categories the project is configured to surface, and only for devices the homeowner attached during consent. Scope and trait enumeration are independent controls, and both must permit an operation for it to succeed.

LEAST PRIVILEGE IN PRACTICE

The resulting grant is narrow in three independent dimensions at once: **by scope** (device interaction only), **by project** (only devices enrolled in this Device Access project), and **by trait** (only the categories the project exposes). Revoking any one of the three cuts access without touching the others.

5.4 The Redirect URI Constraint

This is the architectural crux of the project, and the source of the failure documented in Section 7. It deserves precise treatment because the reasoning generalizes far beyond Nest.

In the authorization code flow, the identity provider returns the code by redirecting the user's browser to a URI registered in advance. Google validates that URI strictly: for a web application client it must be a publicly resolvable HTTPS endpoint, and the value presented at request time must match the registered value *exactly*. This is a security property, not a limitation — it is what prevents an attacker from redirecting an authorization code to a destination they control.

Home Assistant, however, runs at a private address such as `http://192.168.x.x:8123`. That address is not publicly resolvable, has no valid TLS certificate, and is meaningless outside the local network. It can never be registered as an authorized redirect URI. The two requirements are in direct conflict.

There are exactly two ways to resolve it, and the trade-off between them is a real one:

APPROACH	REGISTERED REDIRECT URI	TRADE-OFF
Public redirect bridge (selected)	<code>https://my.home-assistant.io/redirect/oauth</code>	A stable, publicly resolvable TLS endpoint that satisfies Google's validation rules and then hands the authorization response back to whichever local instance URL the browser is configured for. Requires no inbound exposure of the local instance, but does route the callback through a third-party endpoint.

APPROACH	REGISTERED REDIRECT URI	TRADE-OFF
Direct callback	<code><HOME_ASSISTANT_URL>/auth/external/callback</code>	Keeps the callback entirely within infrastructure the operator controls, but requires the instance to be reachable at a public hostname with valid TLS — a reverse proxy, certificate management and an inbound firewall rule.

The public redirect bridge was selected because it satisfies the stated objective of avoiding inbound network exposure. The critical detail — and the one that caused the initial failure — is that the bridge only works if the local instance URL it forwards to is configured correctly and matches the address actually used during setup. When it does not, Google completes the authorization successfully, the browser lands on the bridge, and the handoff to the local instance silently fails. Nothing errors on Google's side, because from Google's perspective nothing went wrong.

THE HONEST CAVEAT

The redirect bridge sits in the path of an authorization response. It is operated by the Home Assistant project, is widely used, and performs the forwarding in the browser rather than server-side — but it is nonetheless a third party in a security-sensitive step. An operator with a public hostname and TLS already in place should prefer the direct callback. This build chose the bridge deliberately, with the trade-off understood rather than overlooked.

5.5 Publishing Status and Token Longevity

The OAuth application was promoted from testing to *in production* publishing status. This is easy to mistake for a cosmetic step, and it is not — it is a durability requirement.

An external OAuth application left in testing status issues refresh tokens that expire after seven days. For an interactive application that is a minor annoyance. For an unattended integration it is fatal: the thermostat silently disconnects roughly weekly and requires manual re-authorization every time. Promoting the application to production removes that expiry and is what makes long-lived, hands-off operation possible.

The application remains unverified, which is correct for this use case. Verification is Google's review process for applications requesting sensitive scopes from the general public. This application has an audience of exactly one — its author — so it stays under the hundred-user cap that applies to unverified external applications, and the only consequence is an unverified-app interstitial during the one-time consent flow.

PUBLISHING STATUS	REFRESH TOKEN LIFETIME	SUITABILITY HERE
Testing	Expires after 7 days	Unusable. Forces weekly manual re-authorization of an unattended integration.
In production (unverified)	No fixed short expiry	Selected. Supports long-lived operation. Capped at 100 users, which is 99 more than required.
In production (verified)	No fixed short expiry	Unnecessary. Verification exists to serve untrusted third-party users; there are none.

5.6 Token Lifecycle

Understanding where each credential lives, and for how long, is what makes the security posture in Section 8 assessable.

CREDENTIAL	LIFETIME	STORED WHERE	NOTES
Authorization code	Seconds	Nowhere — transient	Single-use. Exchanged immediately for tokens over a back channel.

CREDENTIAL	LIFETIME	STORED WHERE	NOTES
Access token	~1 hour	Home Assistant memory	Bearer credential presented on every SDM API call.
Refresh token	Long-lived	Home Assistant local storage	The credential that matters. Must be exercised periodically or the grant lapses.
Client secret	Until rotated	Home Assistant config + Cloud Console	Rotatable without downtime by enabling a new secret before disabling the old.

A SUBTLE OPERATIONAL TRAP

Google revokes authorization if a refresh token goes unused for six months. An integration that relies purely on pushed Pub/Sub events and rarely calls the API can therefore go stale while appearing healthy — events keep arriving right up until the grant lapses. Any command issued from the dashboard exercises the token, so a device that is actually used will not hit this. A purely passive deployment should schedule a periodic API call to keep the grant alive.

5.7 Client Secret Rotation

The OAuth client carries two secrets with distinct lifecycles: an original created when the client was registered, and a replacement created two days later. The replacement is enabled and the original is disabled — a textbook overlap rotation.

The sequence matters. Creating the new secret while the old one is still enabled means there is a window during which both are valid, so the configuration can be updated and verified before the old credential is withdrawn. Reversing the order — disabling first — produces an outage. Google's console explicitly warns that holding more than one active secret increases risk, which is why the old secret is disabled rather than left in place once the new one is confirmed working.

WHY THIS APPEARS IN A PORTFOLIO REPORT

Credential rotation is one of those practices that is universally recommended and rarely performed on personal projects. Doing it here — with correct ordering and with the superseded secret disabled rather than abandoned — demonstrates the operational habit, not just awareness of it. The evidence is visible in Figure 5.

6. Implementation Walkthrough

This section follows the build in the order it was actually performed: establish the boundary, enable the interface, create device trust, configure identity, obtain consent, then verify every layer independently. Each figure is an authentic capture from the live system with identifiers redacted.

6.1 The Cloud Project as Billing and API Boundary

Everything begins with a dedicated Google Cloud project. Using a dedicated project rather than an existing one is a deliberate isolation choice: it scopes billing, quota, API surface and IAM to this integration alone, so nothing here can affect unrelated work and the whole thing can be deleted in one action if needed.

Billing must be attached before anything else will function. This is the step that surprises people, because the integration costs nothing to run — but Google requires an active billing account to *provision* services such as Cloud Pub/Sub, regardless of whether consumption ever leaves the free tier. Figure 1 captures this requirement as an enforcement event rather than as documentation: the notification panel shows a batch of service enablements failing outright with the explicit reason that a billing account for the project was not found and must be enabled for activation to proceed.

Enabling APIs and the billing prerequisite

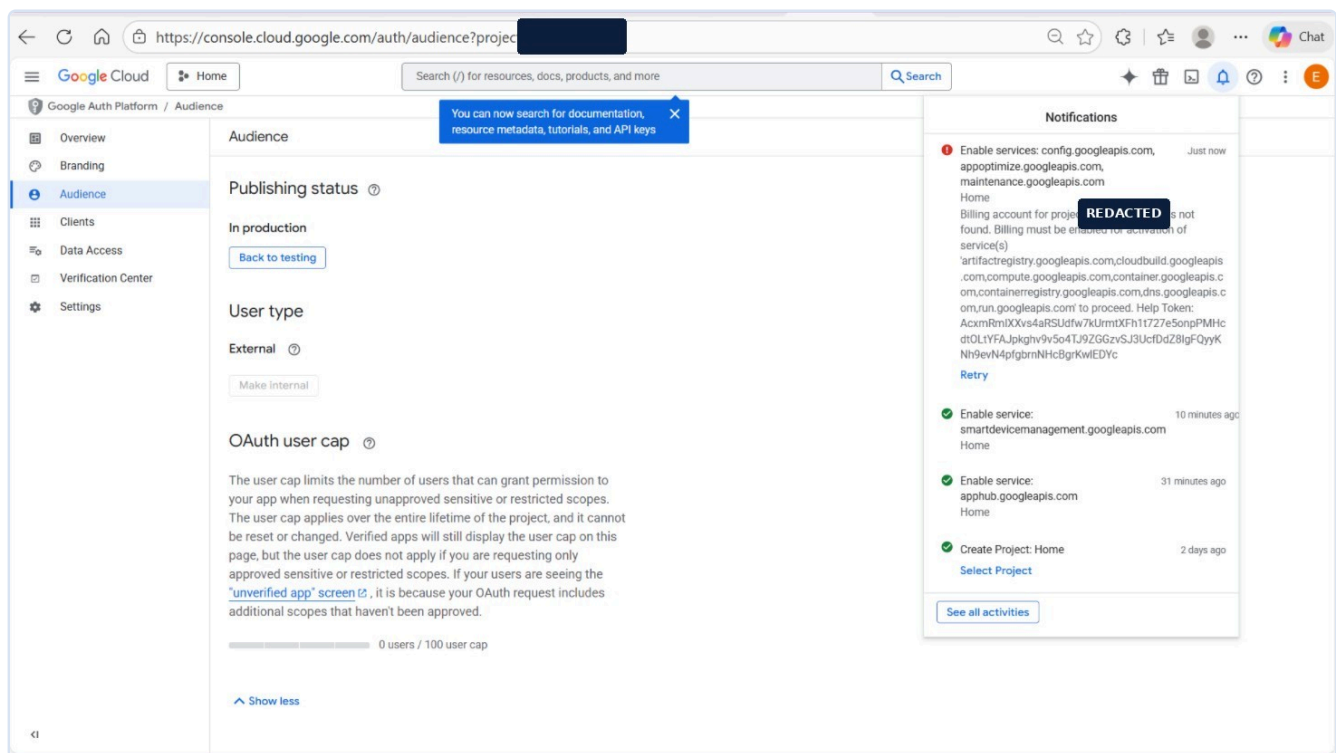


Figure 1. The Google Auth Platform audience view with the activity panel open. The panel records both the billing-blocked enablement attempt and the successful enablement of the Smart Device Management API.

WHAT THIS SCREENSHOT ACTUALLY PROVES

Three things, in one capture. First, the failure notice states plainly that billing must be enabled before a set of services can be activated — the requirement, evidenced rather than asserted. Second, immediately below it, `smartdevicemanagement.googleapis.com` shows as successfully enabled, confirming the first link of the trust chain. Third, the consent audience is already configured as **External** with **In production** publishing status, and the user cap reads **0 users of 100** — the state *before* any authorization grant exists, which becomes the baseline for Figure 4.

6.2 Enabling the Smart Device Management API

With billing attached, the SDM API is enabled explicitly. This API is the authoritative interface to Nest devices and the only supported mechanism for a third-party platform to reach them. Google describes it as permitting select partners to access, control and manage Google and Nest devices programmatically — which is an accurate description of exactly what Home Assistant is doing here.

Smart Device Management API enabled on the project

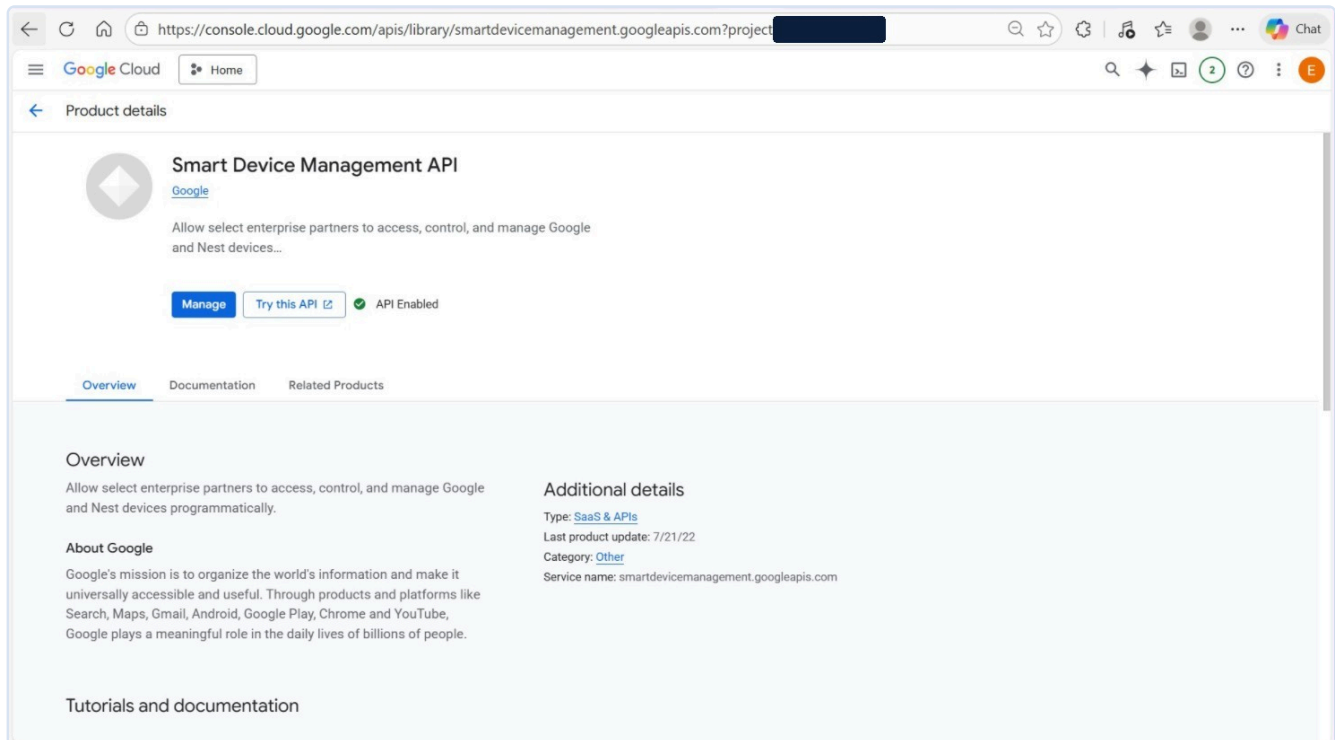


Figure 2. The SDM API product page confirming the enabled state, with the service name that must match the enablement recorded in Figure 1.

WHY ENABLEMENT IS VERIFIED SEPARATELY

API enablement and authorization are independent failures that present identically to the user. A perfectly valid token calling a disabled API returns a permission-style error that looks like an OAuth problem and sends people back to re-check scopes. Confirming enablement on the product page — rather than trusting the earlier notification — eliminates that entire branch of the diagnostic tree before the identity layer is even built.

6.3 The Device Access Project

The Device Access Console is a separate surface from Google Cloud, and enrollment requires a one-time registration fee. This project is the Nest-specific access control layer: it establishes the trust relationship between Nest hardware and a third-party application, and it is where the OAuth client is bound to the device scope.

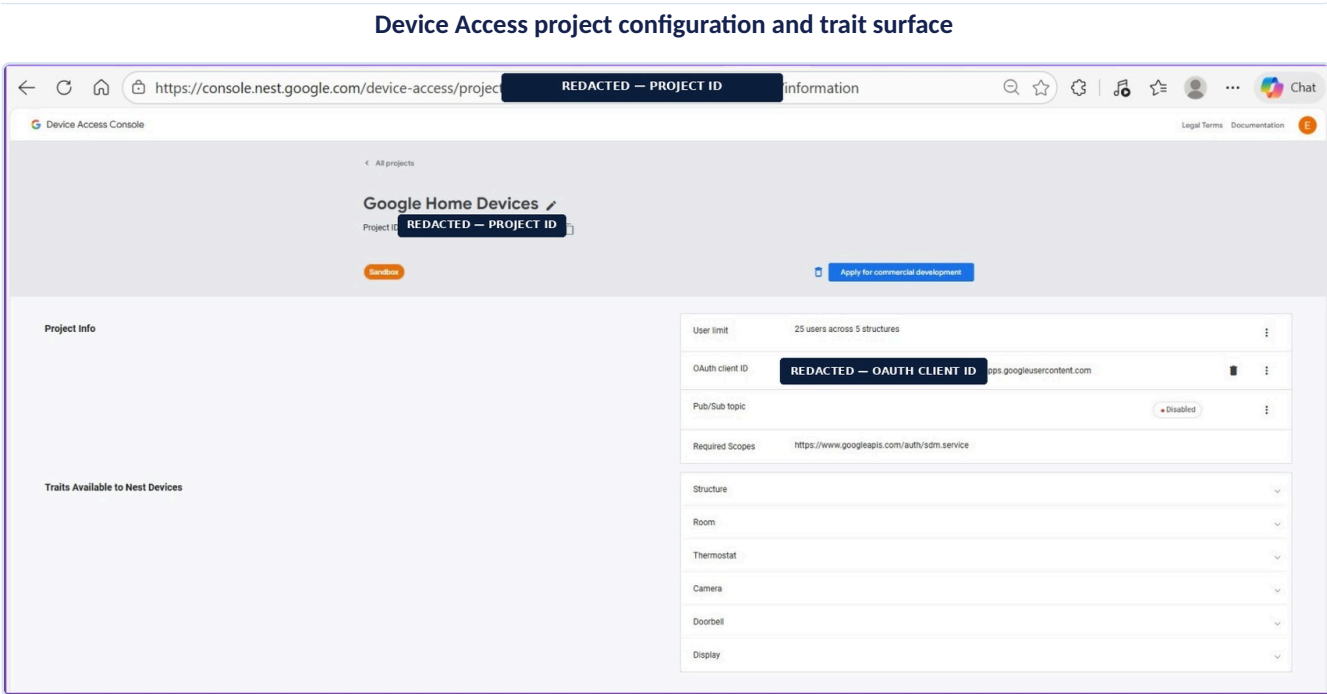


Figure 3. The Device Access project, showing the bound OAuth client ID, the required SDM scope, the sandbox user limits, and the device traits available to the project.

Several details in this capture carry architectural weight:

FIELD	OBSERVED VALUE	SIGNIFICANCE
Project type	Sandbox	Non-commercial development tier. Appropriate for a single-home deployment; commercial distribution would require a separate review process.
User limit	25 users across 5 structures	A hard ceiling enforced by the tier. Comfortably above a single-home requirement, and a useful reminder that this project type is not a distribution channel.
OAuth client ID	Bound (redacted)	Trust chain link 3. This binding is what tells Google which application may request device scopes. Changing it deauthorizes every existing token.
Required scopes	.../auth/sdm.service	The single device scope. Confirms the least-privilege posture described in Section 5.3.
Traits	Structure, Room, Thermostat, Camera, Doorbell, Display	The trait categories the project can surface. Only Thermostat is relevant to this deployment; the others remain unused.

ON THE PUB/SUB TOPIC FIELD

The console records the Google-hosted Pub/Sub topic as disabled for this project. Device Access supports two event-transport models: a legacy Google-hosted topic, and a self-hosted topic living in the operator's own Cloud project. This build uses the self-hosted model, which is consistent with the Pub/Sub resources present in the Cloud project and confirmed in Figure 8. The distinction matters operationally, because the two models are re-provisioned differently if events ever stop arriving — see Section 7.3.

6.4 OAuth Application State and Audience

The consent application is configured with an **External** user type and promoted to **In production**. Section 5.5 covers why: testing status caps refresh token lifetime at seven days, which would disconnect the thermostat weekly.

OAuth application state after the authorization grant

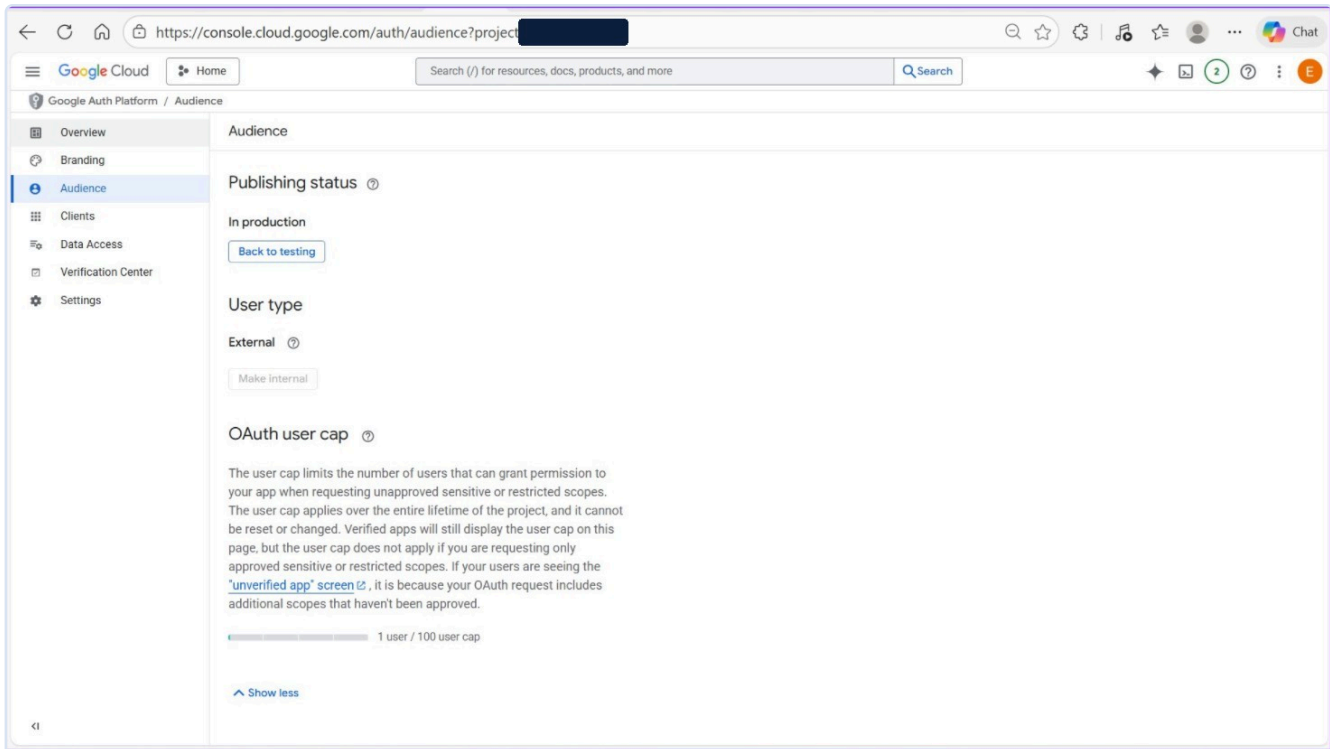


Figure 4. The same audience configuration as Figure 1, now reading one user against the hundred-user cap — the recorded consequence of a completed authorization grant.

A BEFORE-AND-AFTER THAT PROVES THE FLOW COMPLETED

Figures 1 and 4 capture the identical page at two points in time. In Figure 1 the counter reads **0 users / 100**. In Figure 4 it reads **1 user / 100**. That single increment is server-side evidence, recorded by Google rather than claimed by the operator, that a real consent grant was issued and accepted. It is the cleanest possible confirmation that the OAuth flow — the part that initially failed silently — now completes end to end.

6.5 The OAuth Client and Redirect Binding

The client is registered as a **Web application** named for the relying party it represents. Application type is not cosmetic: Google applies different redirect validation rules to each type, and choosing a desktop or installed-application client here produces token exchange failures that are difficult to attribute because they surface only at the final step of the flow.

OAuth client configuration, redirect URI and secret rotation

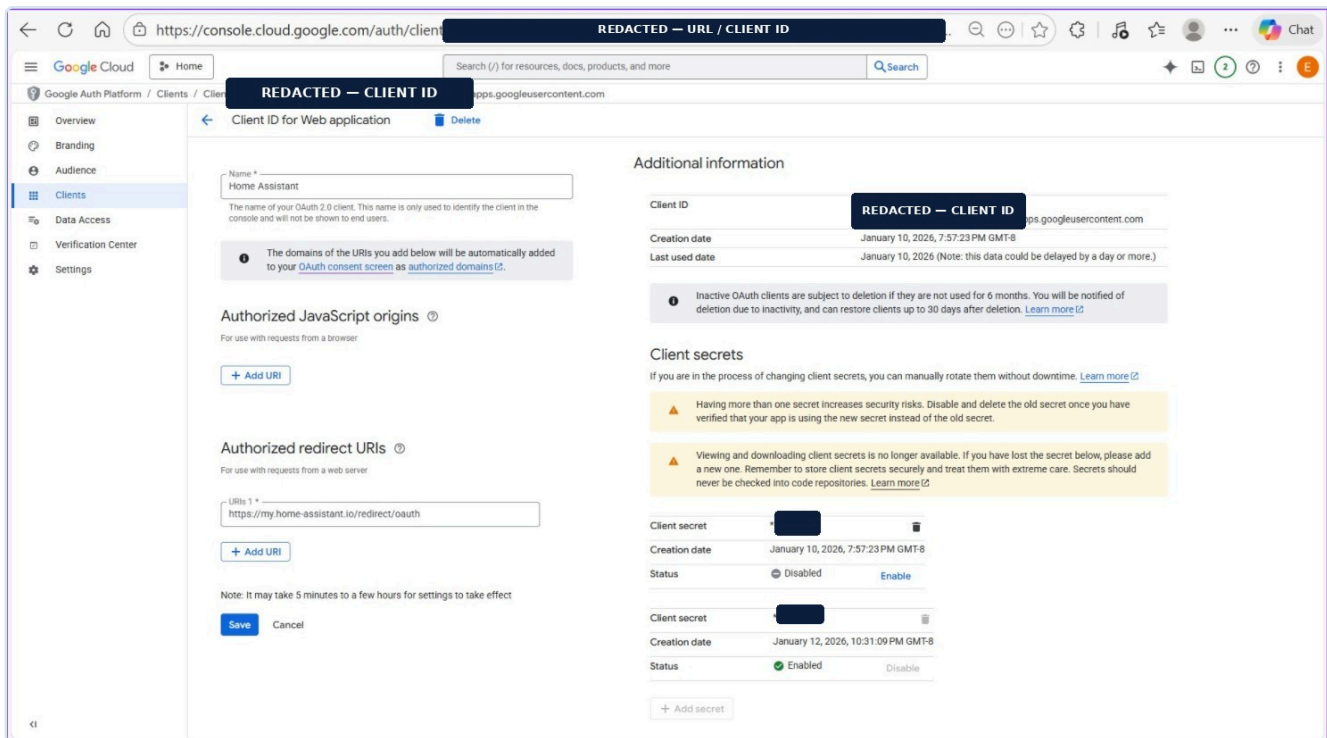


Figure 5. The web application client showing the single authorized redirect URI, and two client secrets — the original disabled, the replacement enabled.

The authorized redirect URI is set to `https://my.home-assistant.io/redirect/oauth`. Section 5.4 explains the reasoning in full: Google requires a publicly resolvable HTTPS callback, Home Assistant lives on a private address, and this endpoint is the bridge that reconciles the two without exposing the local instance to inbound traffic. The URI must match byte for byte between the registration and the request — a trailing slash or a scheme mismatch is sufficient to break the flow.

The right-hand panel documents the credential rotation described in Section 5.7. Two secrets are present: the original, created alongside the client, now **disabled**; and a replacement created two days later, now **enabled**. The console's own warning that multiple active secrets increase risk is visible directly above them, which is precisely why the superseded credential was disabled rather than left active.

READING THE TIMELINE

The creation and last-used timestamps place client registration and the rotation two days apart. That gap is the build window — the client was created, the redirect failure was diagnosed and resolved, and the credential was rotated once the flow was confirmed working. The page also notes that inactive clients are subject to deletion after six months of disuse, which is the same dormancy concern raised for refresh tokens in Section 5.6.

6.6 Verifying the Authorization Grant

Configuration state proves that a system is set up correctly. It does not prove that the homeowner actually granted access. That is recorded separately, on the Nest side, as a partner connection against the home structure.

Partner connection recorded against the home structure

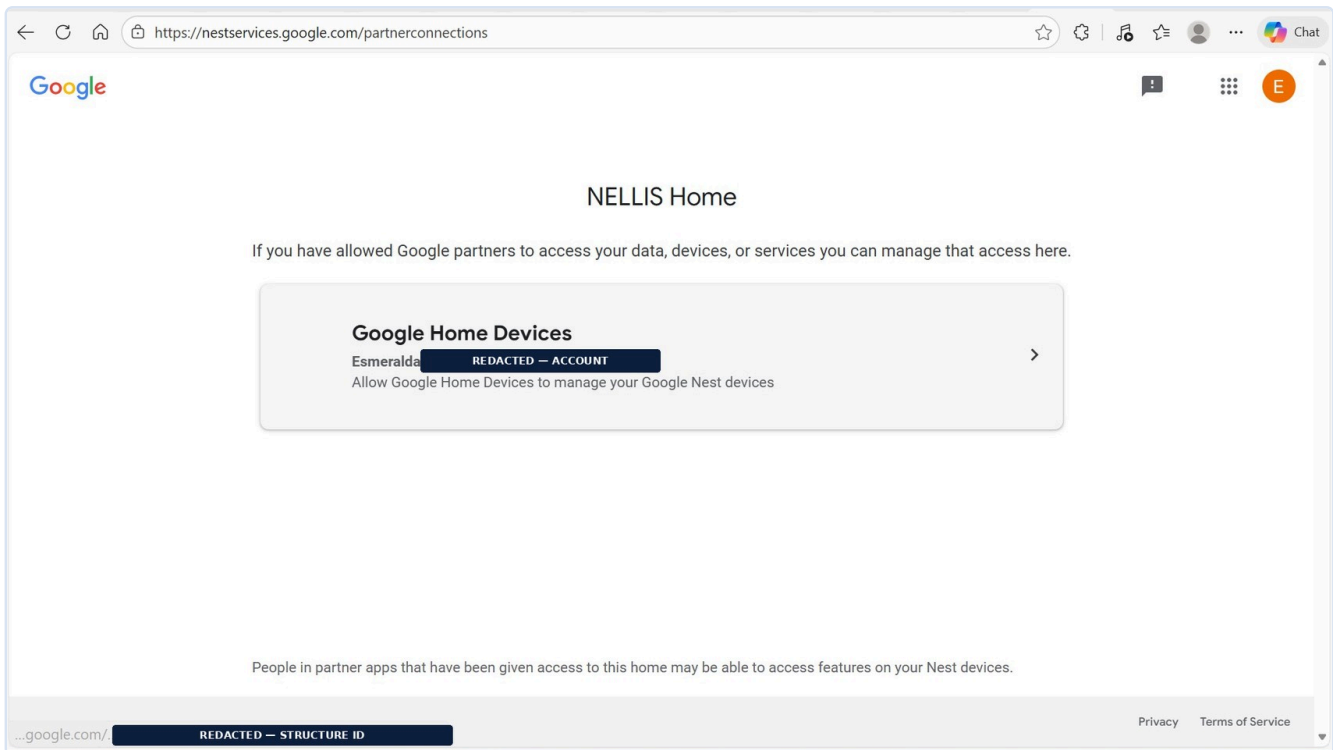


Figure 6. The Nest partner connections view confirming an active grant that permits the registered application to manage Nest devices in the home. Account identifier and structure identifier redacted.

THE FOURTH LINK, FROM THE USER'S SIDE

This page is the homeowner's view of the trust chain, and it is also the revocation control. Access granted here can be withdrawn here, independently of anything in either developer console — which is the correct place for that power to sit. Its presence confirms trust chain link 4 from the opposite direction to Figure 4: Google's identity platform counted a user, and Google's device platform recorded a device grant. Both sides agree.

6.7 Confirming Live API Traffic

With identity and device trust established, the next question is whether the pipeline is actually carrying traffic. The project dashboard answers it directly.

Sustained API request activity on the project

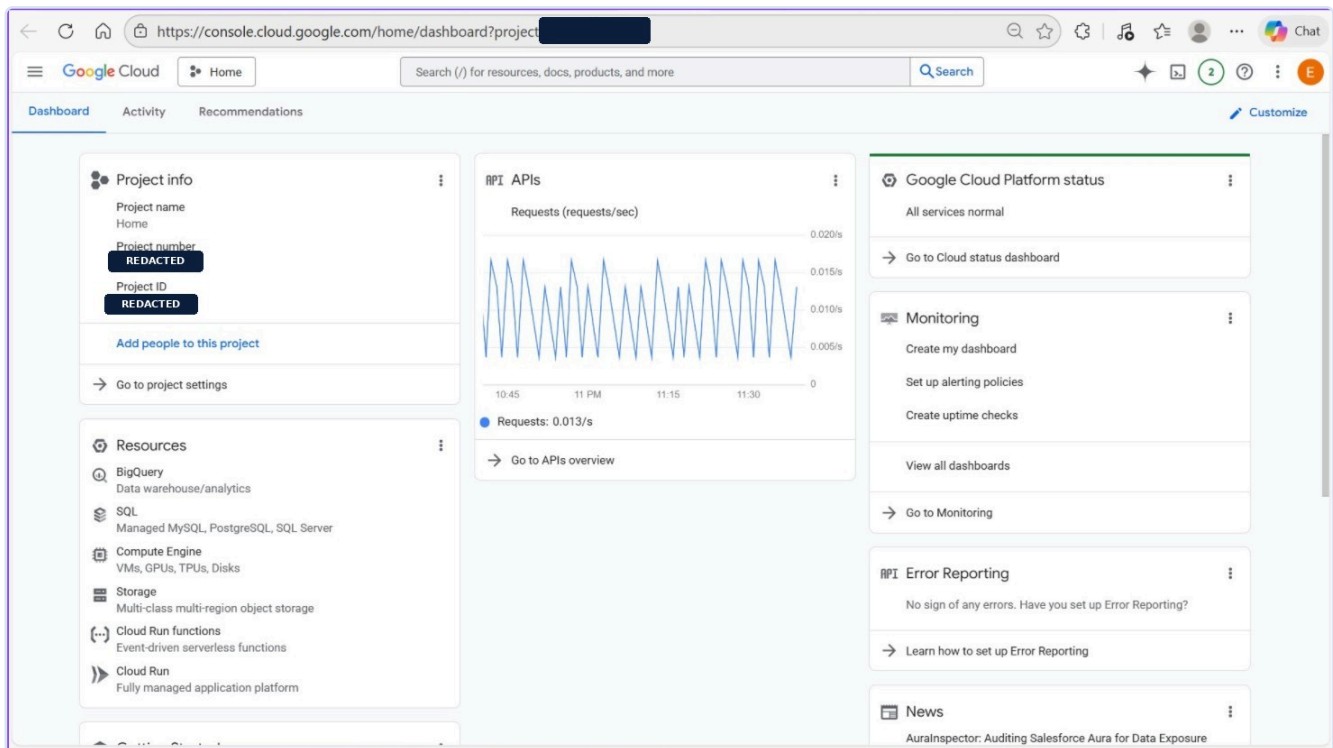


Figure 7. The project dashboard showing a sustained request rate across a 45-minute window, with platform status normal and no errors reported.

The request graph shows a steady oscillation settling around **0.013 requests per second**. That figure is worth converting, because it makes the operating profile concrete:

0.013/s

OBSERVED REQUEST RATE

~47

REQUESTS PER HOUR

~1,100

REQUESTS PER DAY

~34k

REQUESTS PER MONTH

INTERPRETING THE SAWTOOTH

The regular rise-and-fall pattern is characteristic of a periodic maintenance cadence — token refresh, subscription lease renewal and reconciliation — rather than device polling. A polling integration produces a flat line at a fixed interval; an idle event-driven integration produces near-zero traffic punctuated by housekeeping. The important reading is the magnitude: at roughly 47 requests per hour, this is a system doing maintenance, not one repeatedly asking a thermostat what temperature it is. The Error Reporting panel showing no errors across the same window confirms the traffic is succeeding, not retrying.

6.8 The Pub/Sub Event Pipeline

The event pipeline is what makes this integration *Cloud Push* rather than *Cloud Polling*. During the Home Assistant setup flow, a topic and subscription are established in the Cloud project; Google publishes device change events to the topic, and Home Assistant consumes them from the subscription as they arrive.

Configured service integrations in the Cloud project

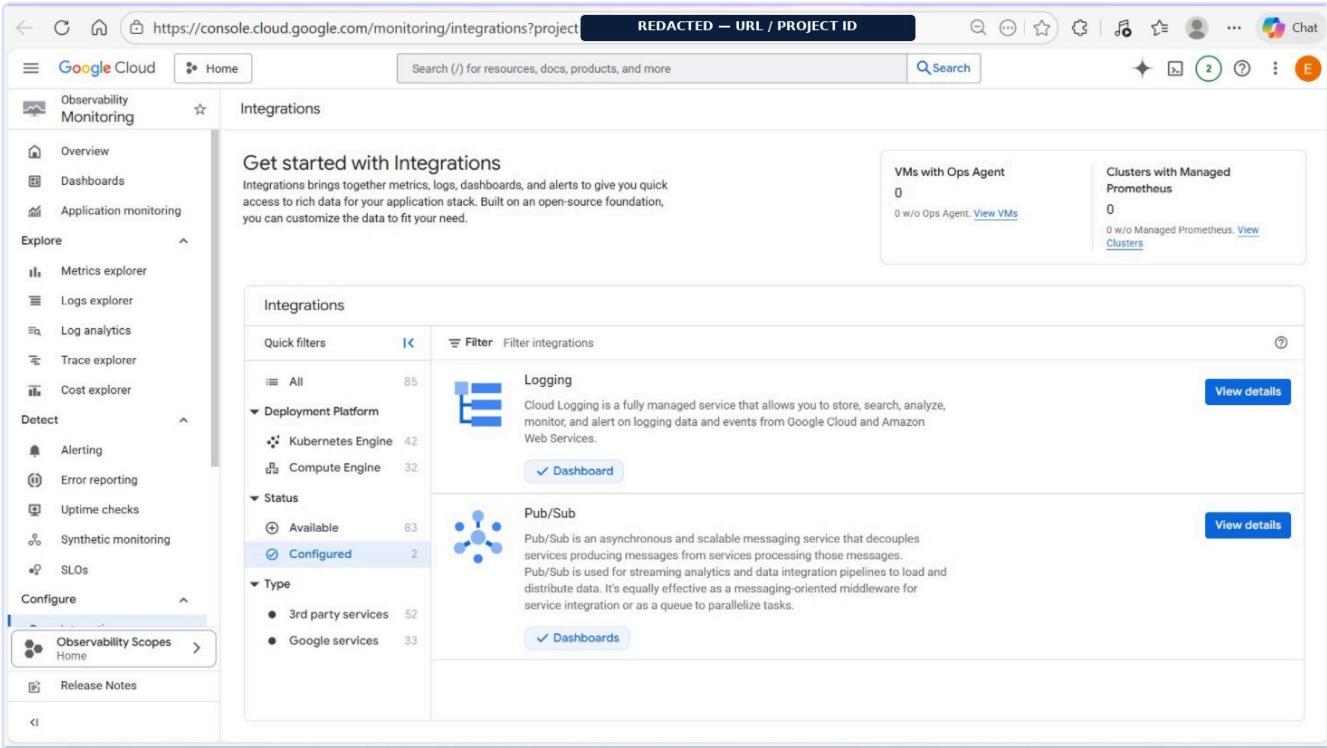


Figure 8. Cloud Monitoring integrations filtered to configured state. Of 85 available integrations, exactly two are configured — Logging and Pub/Sub — matching the intended minimal footprint.

MINIMAL FOOTPRINT, VERIFIED

The filter reads **Configured: 2** against **Available: 83**. That count is a useful architectural assertion in its own right: the project contains no compute, no database and no incidental services — only the messaging primitive the design calls for and the logging that supports it. The Pub/Sub entry showing active dashboards confirms the messaging layer is provisioned and instrumented in the operator's own project, consistent with the self-hosted topic model noted in Section 6.3.

The architectural benefits of this layer are worth stating explicitly, because they are the reason it was built this way rather than with a scheduled fetch:

PROPERTY	CONSEQUENCE FOR THIS SYSTEM
Push delivery	A setpoint change made at the physical thermostat appears on the dashboard within seconds, with no request issued by the consumer.
Zero idle cost	Nothing is transmitted while nothing changes. A polling design pays its full request budget whether or not the device is doing anything.
Decoupling	The producer does not know or care whether a consumer is currently connected.
Buffering	If Home Assistant restarts or the network drops, messages accumulate in the subscription and are delivered on reconnection rather than lost.
Free-tier headroom	Event volume for a single thermostat is orders of magnitude below the monthly free allowance — see Section 9.

6.9 End-to-End Validation on the Live System

The final verification is the one that matters to a user: the device appears in Home Assistant, reports accurate live state, and is controllable. Every preceding layer could be configured correctly and this could still fail; conversely, this working means the entire chain is intact.

Google Nest Thermostat live in the Home Assistant control plane

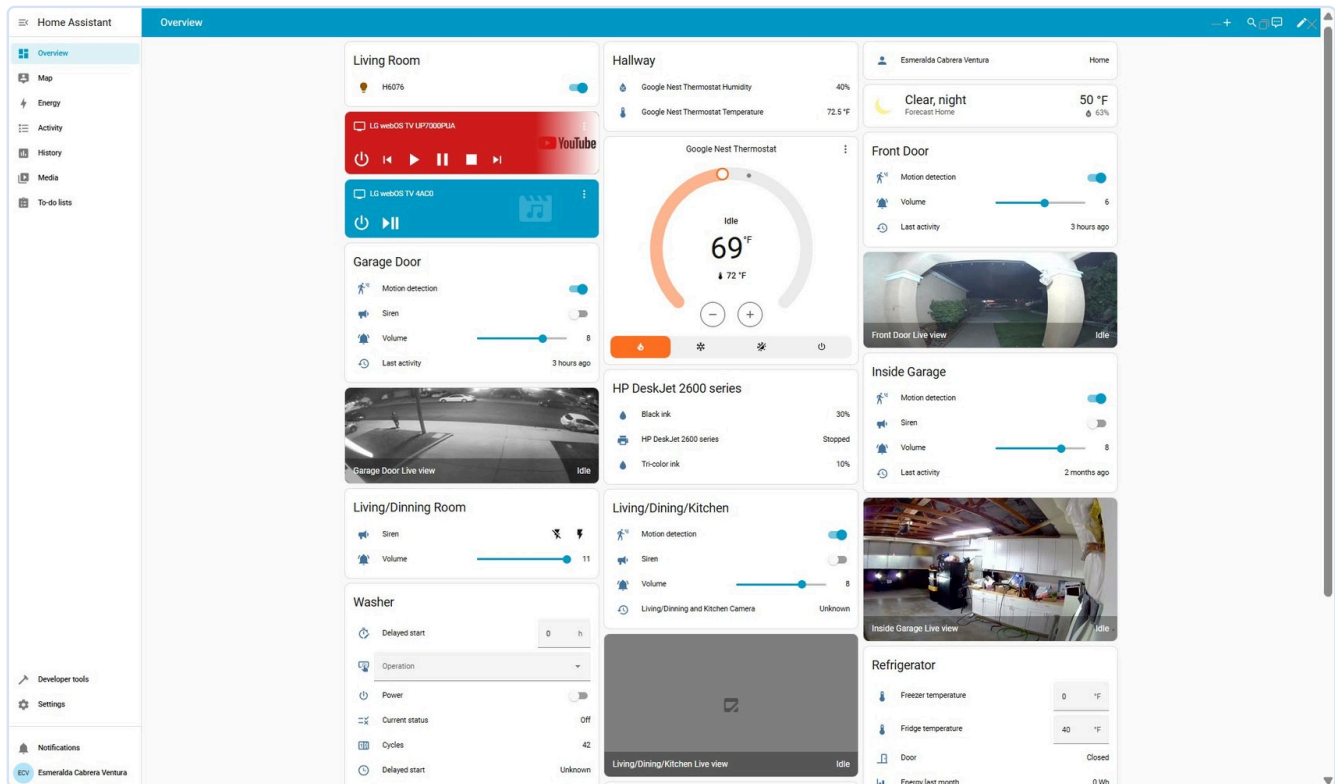


Figure 9. The Home Assistant dashboard with the Nest Thermostat rendered as a controllable climate entity, alongside the discrete temperature and humidity sensor entities it exposes.

The thermostat card reads a **69°F** target against a **72°F** ambient temperature with the system in an **Idle** state — internally consistent, since a heating setpoint below the current ambient temperature is exactly the condition under which the system should not be calling for heat. The mode selector beneath the dial exposes heat, cool, heat-cool and off, confirming the control plane is live and not merely displaying cached data.

The integration surfaces the device as more than a single control. Alongside the climate entity, discrete sensor entities report **72.5°F** and **40%** relative humidity. The half-degree resolution on the sensor against the whole-degree display on the climate card is the expected behaviour: the climate entity rounds for presentation while the sensor entity preserves the raw reported value. That distinction matters for the next phase — an AI agent reasoning over temperature trends wants the unrounded series, not the display value.

THE WIDER CONTEXT

The Nest thermostat is one card among many on this dashboard — cameras, media players, appliances and lighting from unrelated vendors are all present. That is the strategic payoff of the whole exercise. Once the device is normalized into Home Assistant's entity model, it stops being a Google device and becomes an addressable component of a vendor-neutral control plane, able to participate in automations alongside hardware that shares no ecosystem with it.

7. Failure Analysis & Troubleshooting

7.1 The Silent Authentication Failure

The first authorization attempt failed in the most instructive way possible: it did not error. The Google consent screen appeared, the grant was accepted, the browser redirected — and Home Assistant simply never received a token. No error banner, no log entry pointing at a cause, nothing on Google's side indicating a problem.

That silence is diagnostic in itself. It rules out a large part of the search space immediately:

HYPOTHESIS	VERDICT	REASONING
Invalid client credentials	Eliminated	The consent screen rendered. Google will not display consent for an unrecognized client.
Insufficient scope	Eliminated	The consent screen listed the requested permission and the grant was accepted.
API not enabled	Eliminated	API enablement failures occur at call time, after tokens exist. No token was ever issued.
Device not enrolled	Eliminated	Enrollment problems present as an empty device list following a successful authorization, not as a missing token.
Redirect boundary mismatch	Root cause	The only failure mode that completes successfully on the identity provider's side and produces nothing on the relying party's side. Google's work finished correctly; the response never reached its destination.

The reasoning is structural rather than empirical. Everything before the redirect demonstrably worked, and everything after it demonstrably did not. The fault therefore lies precisely at the handoff — the one point in the flow where responsibility transfers from Google to the local instance. As covered in Section 5.4, Google validates the callback strictly and considers its obligation discharged once it redirects the browser to the registered URI. Whether anything is listening at the other end is not Google's concern, and not something it can report on.

RESOLUTION

Aligning the redirect boundary resolved it: registering the exact publicly resolvable callback URI on the OAuth client, and ensuring the bridge forwards to the same local instance address actually used during setup. The two values must agree — a mismatch in scheme, port, host or trailing slash reproduces the identical silent failure. Figure 4's incremented user counter is the confirmation that the corrected flow completes.

7.2 Failure Taxonomy

Each failure mode encountered or anticipated during the build, mapped to the trust chain link that causes it. The value of this table is that the symptom rarely points at the responsible layer.

SYMPTOM	LINK	ROOT CAUSE	RESOLUTION
Service enablement rejected	1	No billing account attached to the project	Attach an active billing account, then retry enablement. Consumption may still be zero.
API calls rejected despite a valid token	1	SDM API not enabled on the project	Enable <code>smartdevicemanagement.googleapis.com</code> and confirm on the product page.

SYMPTOM	LINK	ROOT CAUSE	RESOLUTION
Consent completes, no token arrives	2	Redirect URI mismatch or unreachable callback destination	Register the exact public callback URI; verify the forwarding target matches the address used at setup.
Explicit redirect URI mismatch error	2	Registered URI differs from the one presented at request time	Match byte for byte, including scheme, port and trailing slash.
Disconnection roughly every seven days	2	OAuth application left in testing publishing status	Promote the application to in-production status.
Token exchange fails at the final step	2	Wrong OAuth client type registered	Use a web application client; desktop and installed types apply different redirect rules.
Consent grant lands on the wrong account	4	Multiple Google accounts active in the browser session	Complete the flow in a private window with only the target account signed in.
Authorization succeeds, no devices returned	3	OAuth client not bound in the Device Access project, or devices not attached during consent	Verify the client ID binding in the Device Access Console and re-run consent, selecting the devices.
Previously working integration stops abruptly	3	OAuth client ID changed in the Device Access project	Changing the bound client deauthorizes existing tokens. Re-authorize after any such change.

7.3 Time-Delayed Failure Modes

The failures that matter most in an unattended system are the ones that appear weeks or months after a successful build, when the original context has been forgotten. Three apply here, and all three share a cause: dormancy.

TRIGGER	HORIZON	CONSEQUENCE AND MITIGATION
Subscription inactivity	31 days	Pub/Sub subscriptions are subject to deletion after a period of inactivity by default. If the subscription disappears, the integration fails to start because it looks for a subscription that no longer exists. Mitigation: edit the subscription's expiration policy so it does not expire.
Refresh token dormancy	6 months	An integration that consumes pushed events but rarely calls the API can have its authorization revoked for non-use. Mitigation: ensure the token is exercised periodically — normal device control is usually sufficient.
OAuth client inactivity	6 months	Google's console notes that inactive OAuth clients are subject to deletion, with a limited restoration window after removal. Mitigation: the same periodic activity that protects the refresh token protects the client.

THE LESSON WORTH CARRYING FORWARD

All three of these are silent until the moment they are not. None produces a warning, and none is visible on a dashboard that shows the system working today. Documenting them at build time — while the reasoning is fresh — is what separates a system that was made to work from a system that is designed to keep working.

8. Security Posture

8.1 What the Design Protects

CONTROL	EFFECT
Scope minimization	A single device scope. A compromised token grants thermostat interaction, not account access — no mail, files, contacts or location history.
Project separation	Device trust lives in a different project from cloud resources. Revoking device access does not disturb the Cloud project, and vice versa.
No inbound exposure	The local instance accepts no unsolicited inbound connections. Both cloud paths are outbound-initiated from the local side.
No device credentials locally	Home Assistant never holds Nest device credentials — only an OAuth refresh token scoped to a delegated grant, revocable by the homeowner at any time.
Independent revocation	Access can be withdrawn from the partner connections page without touching either developer console.
Secret rotation	Client secrets are rotated with overlap and the superseded credential disabled, not abandoned in an active state.
Minimal attack surface	No compute, no database, no reverse proxy, no open ports. Two configured services in the entire project.

8.2 Threat Model

THREAT	EXPOSURE	ASSESSMENT
Authorization code interception	Low	The code is single-use, short-lived, and exchanged over a back channel. Google's strict redirect validation is the primary defence against redirection to an attacker-controlled endpoint.
Client secret disclosure	Moderate	The secret is stored in local configuration. Disclosure alone is insufficient without a valid refresh token, and rotation is straightforward and already exercised.
Refresh token theft	Moderate	The highest-value credential in the system. Compromise permits thermostat control until revoked — bounded by scope to device interaction, and revocable from the homeowner's partner connections page.
Redirect bridge compromise	Low but non-zero	A third-party endpoint sits in the callback path during the one-time setup flow. It is not involved in steady-state operation. See Section 8.3.
Local instance compromise	Moderate	An attacker with the local host gains the refresh token and control of every integration on it. Mitigated by the absence of inbound exposure; local host hardening is the operator's responsibility.
Cloud project takeover	Low	Would require compromise of the Google account itself. Protected by account-level controls outside this integration's boundary.
Event stream tampering	Low	Pub/Sub delivery is authenticated and transport-encrypted between Google and the consumer.

8.3 Residual Risk — An Honest Assessment

Three residual risks are worth stating plainly rather than omitting.

The redirect bridge. The chosen callback routes the authorization response through an endpoint operated by the Home Assistant project. The forwarding happens in the browser rather than server-side, it occurs only during initial setup, and it is the documented approach for local installations — but it is a third party in a security-sensitive step. An operator who already has a public hostname with valid TLS should prefer the direct callback described in Section 5.4. This build accepted the trade-off knowingly, in exchange for not exposing the local instance inbound.

Local credential storage. The refresh token and client secret live in configuration on the local host. Their security is therefore the security of that host. This is inherent to a self-hosted control plane rather than a flaw in the integration, but it means host hardening, patching and backup encryption are load-bearing controls, not optional hygiene.

Unverified application status. The OAuth application is in production but unverified, which is appropriate for an audience of one and imposes only a consent-time interstitial. It would not be appropriate if the application were ever distributed, and the hundred-user cap is a hard technical boundary on that, not merely a policy one.

OVERALL POSTURE

For a residential integration, the posture is strong: a narrowly scoped, individually revocable, user-delegated grant with no inbound network exposure and no long-lived device credentials on local hardware. The dominant risk is not the cloud path — it is physical or network compromise of the local host, which is the same risk any self-hosted platform carries and is addressed outside this integration's boundary.

9. Cost Analysis

A stated objective was to deliver real-time Nest integration without a recurring subscription. The cost structure confirms it, and the reasoning is worth showing rather than asserting.

ITEM	COST	BASIS
Device Access registration	One-time fee	A single non-recurring charge required to enrol in the Device Access program. The only unavoidable cost in the project.
Billing account	\$0	Required to <i>provision</i> Pub/Sub and SDM. Attachment is mandatory; consumption is not.
Smart Device Management API	\$0	No per-request charge at the volumes involved. Observed traffic is roughly 34,000 requests per month.
Cloud Pub/Sub	\$0	Well inside the monthly free allowance — see the volume estimate below.
Cloud Monitoring & Logging	\$0	Baseline metrics and logs for a project at this volume fall within free-tier limits.
Compute / storage	\$0	None provisioned. The architecture has no compute layer.
Recurring monthly total	\$0	All ongoing consumption sits within free-tier allowances.

Pub/Sub volume estimate

Pub/Sub's free allowance is measured in message throughput per month, denominated in gibibytes. A single thermostat is nowhere near it, and it is worth quantifying the margin:

QUANTITY	ESTIMATE
Observed API request rate	~0.013 requests/second (Figure 7)
Implied monthly request volume	~34,000 requests
Typical device event message size	on the order of 1–2 KB
Implied monthly message throughput	well under 100 MB
Monthly free allowance	10 GiB
Headroom	Two orders of magnitude below the free tier

THE ECONOMIC ARGUMENT

The event-driven design is not only architecturally cleaner than polling — it is what keeps the cost at zero. A polling integration consumes its full request budget continuously regardless of whether the thermostat is doing anything. This design transmits only on change, so an idle house generates almost no traffic. Real-time responsiveness and near-zero cost are, in this case, the same design decision.

10. Observability & Operations

An integration that runs unattended needs to be answerable to a simple question: is it still working? Three signals cover it, and all three are visible in the evidence already presented.

SIGNAL	SOURCE	WHAT IT TELLS YOU
API request rate	Project dashboard (Figure 7)	Traffic is flowing. A drop to zero means the grant has lapsed, the token has expired, or the local consumer is down.
Error reporting	Project dashboard (Figure 7)	Requests are succeeding rather than retrying. A rise in errors alongside steady request volume indicates authorization or quota trouble.
Pub/Sub dashboards	Cloud Monitoring (Figure 8)	Messages are being published and acknowledged. A growing unacknowledged backlog means the local consumer has stopped reading.
Entity freshness	Home Assistant (Figure 9)	The end-to-end proof. A stale last-updated timestamp on the climate entity is the first symptom a user actually notices.

Recommended alerting

The project currently has monitoring integrations configured but no alerting policies defined. Given the time-delayed failure modes in Section 7.3, three policies would provide meaningful coverage:

- **API request rate falls to zero** over a multi-hour window — the earliest reliable signal that the grant has lapsed or the consumer has stopped.
- **Unacknowledged message count grows** beyond a small threshold — indicates the local consumer is offline while Google is still publishing.
- **Subscription age approaches the inactivity horizon** — a guard against the 31-day expiry described in Section 7.3, unless the expiration policy has been disabled outright.

AN OPERATIONAL GAP, STATED HONESTLY

The system is well instrumented but not yet alerted. Every signal needed to detect failure is being collected; none of it currently pages anyone. For a residential thermostat that is a reasonable position — the failure is visible on the dashboard and the consequence is inconvenience. It is recorded here because the distinction between *observable* and *monitored* is one that matters far more in production systems, and the gap should be named rather than glossed over.

11. Next Phase: The AI Agent Control Layer

The integration delivered so far produces something more valuable than a controllable thermostat: a normalized, event-driven, authenticated representation of the physical environment, sitting on hardware the operator controls. That is precisely the substrate an autonomous agent needs. The next phase places an AI agent above Home Assistant as an intelligent orchestration layer.

11.1 The Architectural Boundary

The single most important decision in this phase is what the agent is *not* given. The agent receives no cloud credentials, no OAuth tokens, no client secrets and no direct access to the SDM API. It interacts exclusively with Home Assistant's local interface, consuming normalized state and issuing intent-based actions that Home Assistant executes through its existing authenticated pipeline.

DESIGN CHOICE	CONSEQUENCE
No credentials reach the agent	The credential blast radius is unchanged from the current build. An agent compromise cannot yield an OAuth token, because the agent never holds one.
Commands traverse the existing pipeline	Every action the agent takes is subject to the same authentication, scope limits and audit trail as a human clicking the dashboard. There is no privileged side channel.
The agent consumes normalized entities	It reasons over a vendor-neutral model, not a Google-specific schema, so it generalizes across hardware without knowing which vendor supplies what.
Revocation stays where it was	Withdrawing the partner connection disables the agent's reach over Nest devices as a side effect. No separate revocation path is introduced.

WHY THIS BOUNDARY IS THE RIGHT ONE

The tempting shortcut is to give the agent API access directly — it is fewer moving parts and feels more capable. It is also strictly worse on every axis that matters. Direct access would duplicate credential storage, bypass the audit trail, couple the agent to a vendor-specific schema, and widen the blast radius of a component whose behaviour is the least predictable in the system. Constraining the agent to the local control plane costs nothing in capability and keeps the security model already established completely intact.

11.2 What the Agent Can Observe

Home Assistant already exposes everything the agent needs, and the current build demonstrates the shape of it. From the thermostat alone:

SIGNAL	FORM	AGENT USE
Climate entity	Setpoint, mode, action state	Current intent and whether the system is actively conditioning.
Temperature sensor	Unrounded value, event-driven	The series to reason over. Preserves resolution the climate card rounds away.
Humidity sensor	Percentage, event-driven	Comfort modelling, which temperature alone cannot express.
State history	Time series, local	Thermal response modelling — how fast the space actually heats and cools.
Event stream	Push, on change	Reactive triggers without polling, inherited directly from the Pub/Sub design.

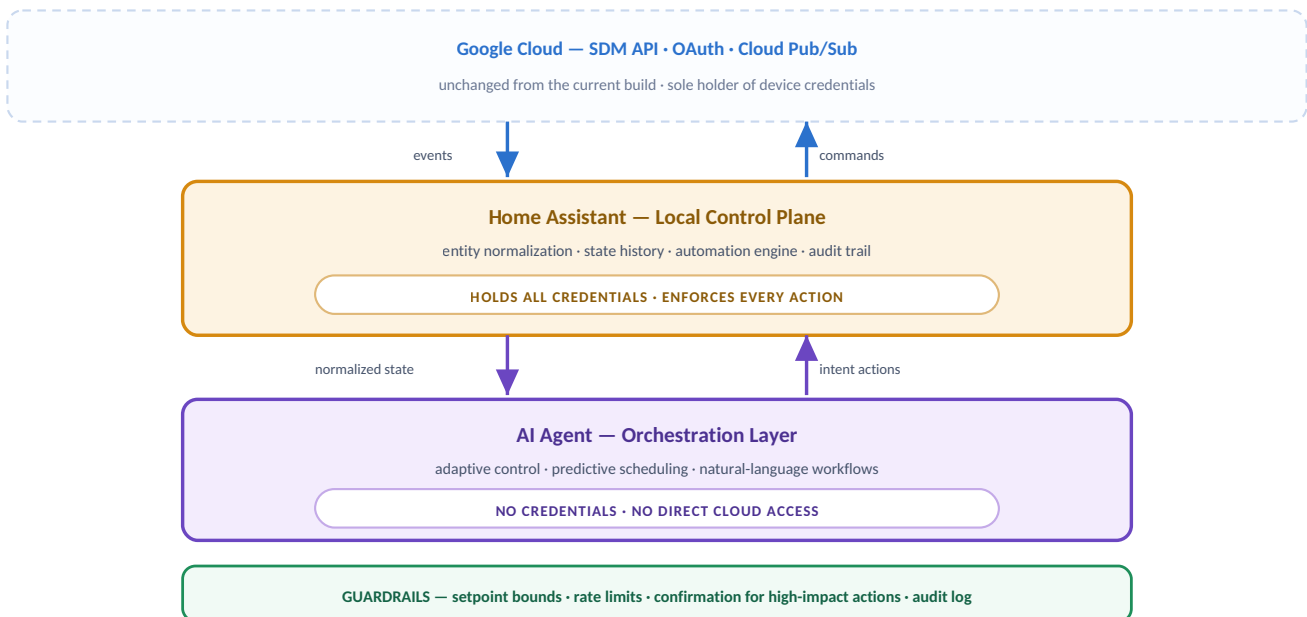
SIGNAL	FORM	AGENT USE
Adjacent entities	Occupancy, weather, lighting, media	Cross-domain context. Section 6.9 shows how much of it is already present.

The forecast and occupancy data visible on the dashboard alongside the thermostat are what make genuinely adaptive behaviour possible. A control loop that sees only the current temperature can do little more than a conventional thermostat. One that sees the temperature series, the outdoor forecast, the occupancy signal and the thermal response history can anticipate rather than react.

11.3 Capability Model

CAPABILITY	BEHAVIOUR
Adaptive setpoint control	Adjust targets from occupancy, outdoor forecast and observed comfort patterns rather than a fixed schedule, within operator-defined bounds.
Predictive scheduling	Learn the thermal response of the space and begin conditioning early enough to hit a target at a target time, rather than reacting once the setpoint is crossed.
Natural-language workflows	Translate instructions such as "keep the house cooler while I'm away this week" into concrete, bounded entity actions.
Cross-domain reasoning	Combine thermostat state with occupancy, weather and other entities to make decisions no single-device integration could reach.
Anomaly detection	Flag divergence from learned behaviour — a space heating far slower than its own history predicts is a maintenance signal, not a comfort one.
Explanation	Report the reasoning behind each action in plain language, so the operator can audit intent rather than merely observing outcomes.

11.4 Proposed Architecture



The important structural feature is that the agent is the *lowest-privilege* component in the system despite being the highest-level one. Everything above the amber box in the diagram is unchanged from the delivered build; the agent attaches to the local control plane and inherits its constraints rather than acquiring new ones.

11.5 Guardrails and Safety Design

An agent with authority over a physical heating and cooling system needs bounded authority. Six controls are specified for the next phase:

GUARDRAIL	PURPOSE
Absolute setpoint bounds	Hard minimum and maximum enforced by the control plane, not by the agent. No reasoning path can produce a target outside the safe range.
Rate limiting	A floor on the interval between changes. Prevents oscillation and protects HVAC equipment from short-cycling.
Confirmation for high-impact actions	Large deviations or extended schedule changes require explicit approval rather than executing autonomously.
Dry-run mode	The agent proposes actions without executing them during an initial evaluation period, so its judgement can be assessed before it is trusted.
Full audit log	Every action recorded with its reasoning. Enables retrospective review and is the basis for tuning behaviour.
Manual override precedence	A human adjustment always wins and suspends agent control for a defined window. The operator is never in a contest with the automation.

ENFORCEMENT BELONGS BELOW THE AGENT

Each of these is enforced by the control plane, not implemented inside the agent. A guardrail that the agent enforces on itself is a guardrail that fails whenever the agent misbehaves — which is precisely the condition it exists to handle. Bounds checked at the point of execution hold regardless of what the reasoning layer concludes, and that is the difference between a safety control and a stated intention.

12. Outcomes & Skills Demonstrated

The delivered system meets every objective set out in Section 2.2. A Google Nest Thermostat is fully integrated into a self-hosted Home Assistant instance, updating in near real time through an event-driven pipeline, secured by a narrow OAuth grant, at zero recurring cost.

OBJECTIVE	STATUS	EVIDENCE
Dedicated Cloud project as security and billing boundary	Delivered	Figures 1, 7 — billing enforcement, minimal footprint
SDM API enabled as the authoritative device interface	Delivered	Figures 1, 2 — enablement event and product page confirmation
OAuth 2.0 identity layer durable for unattended operation	Delivered	Figures 4, 5 — in-production status and registered client
Redirect boundary conflict resolved	Delivered	Figure 5 redirect URI; Figure 4 user counter increment
Device trust isolated in a separate project	Delivered	Figure 3 — client binding, scope, traits
Polling replaced with event-driven messaging	Delivered	Figure 8 — Pub/Sub configured; Figure 7 — request profile
Zero recurring cost	Delivered	Section 9 — all consumption within free-tier allowances
Independent verification at every layer	Delivered	Nine captures spanning enablement through rendered state
Clean seam for an AI orchestration layer	Specified	Section 11 — architecture, capability model and guardrails defined

Capabilities exercised

DOMAIN	DEMONSTRATED THROUGH
Cloud identity engineering	OAuth 2.0 authorization code flow, publish-state management, scope minimization and cross-console trust delegation.
Event-driven architecture	Pub/Sub topic and subscription design, push-versus-poll analysis, and consumer decoupling with buffered delivery.
Hybrid cloud-to-local integration	Joining a public identity provider to a private, non-routable service without inbound exposure.
Systematic troubleshooting	Diagnosing a silent authentication failure by eliminating hypotheses structurally rather than by trial and error.
Security engineering	Least-privilege scope design, credential rotation with correct overlap ordering, threat modelling and residual-risk assessment.
Cost engineering	Delivering real-time capability within free-tier allowances, with volume analysis to substantiate it.
Operational maturity	Documenting time-delayed failure modes, specifying alerting policies and naming the observability gap.
Forward architecture	Specifying an AI orchestration layer whose credential boundary leaves the existing security model intact.

THE TRANSFERABLE RESULT

The thermostat is the smallest part of this project. What was built is a working instance of a pattern that recurs constantly in enterprise environments: a third-party platform holds the data, a private system needs it in real time, and the two must be joined through federated identity and asynchronous messaging without opening the perimeter. Every technique here transfers directly.

Appendix A — Reference Tables

A.1 Scopes and service identifiers

IDENTIFIER	PURPOSE
<code>smartdevicemanagement.googleapis.com</code>	The SDM service name enabled on the Cloud project.
<code>https://www.googleapis.com/auth/sdm.service</code>	The device scope required by the Device Access project.
<code>https://www.googleapis.com/auth/pubsub</code>	Requested by the setup flow to create and consume the event subscription.
<code>https://my.home-assistant.io/redirect/oauth</code>	The registered authorized redirect URI — the public bridge to the local instance.
<code><HOME_ASSISTANT_URL>/auth/external/callback</code>	The alternative direct callback, for instances with a public hostname and valid TLS.

A.2 Configuration summary

SETTING	VALUE AS BUILT
Cloud project role	Dedicated security, billing and API boundary
Billing	Active — required for provisioning, not for consumption
OAuth user type	External
OAuth publishing status	In production (unverified)
OAuth user cap	1 of 100 consumed
OAuth client type	Web application
Authorized redirect URIs	1
Client secrets	2 present — 1 enabled, 1 disabled (rotated)
Device Access tier	Sandbox — 25 users across 5 structures
Device traits available	Structure, Room, Thermostat, Camera, Doorbell, Display
Configured Cloud integrations	2 — Cloud Logging, Cloud Pub/Sub
Observed API request rate	~0.013 requests/second
Home Assistant IoT class	Cloud Push
Entities exposed	Climate, temperature sensor, humidity sensor

A.3 Trust chain quick reference

LINK	RELATIONSHIP	FAILURE SYMPTOM IF BROKEN
1	Cloud project → SDM API	Valid tokens, but every API call is rejected.
2	Cloud project → OAuth client	Flow completes on Google's side; no token ever reaches the relying party.
3	Device Access project → OAuth client	Authorization succeeds; the device list comes back empty.
4	Homeowner → application	No partner connection recorded; no device access at all.

Appendix B — Glossary

Access token	A short-lived bearer credential presented on each API call. Expires within roughly an hour.
Authorization code flow	The OAuth 2.0 grant type in which a user-delegated code is exchanged over a back channel for an access token and a refresh token.
Client secret	The confidential credential proving an OAuth client's identity during token exchange. Rotatable with overlap.
Cloud Push	Home Assistant's classification for integrations that receive state changes pushed from a cloud service, as opposed to Cloud Polling.
Consent audience	The population permitted to authorize an OAuth application. External means any Google account, subject to the user cap.
Device Access	Google's program governing third-party access to Nest devices. Uses a separate console and project type from Google Cloud.
Entity	Home Assistant's normalized representation of a single device capability — a thermostat surfaces climate, temperature and humidity entities.
Partner connection	The record of a homeowner's grant permitting an application to manage their Nest devices. Also the revocation control.
Publishing status	Whether an OAuth application is in testing or production. Testing caps refresh token lifetime at seven days.
Pub/Sub subscription	The endpoint from which a consumer reads messages published to a topic. Buffers during consumer down-time.
Pub/Sub topic	The named channel to which device change events are published.
Redirect URI	The pre-registered destination to which an identity provider returns the authorization response. Validated strictly and exactly.
Refresh token	The long-lived credential used to obtain new access tokens without user interaction. The operationally critical credential.
Scope	The permission requested during authorization, bounding what a token may do.
SDM API	Smart Device Management API — the authoritative and only supported third-party interface to Nest devices.
Trait	A category of device capability that a Device Access project may expose, such as Thermostat or Camera.
Trust chain	In this report, the four linked relationships that together permit an application to reach a homeowner's devices.

Appendix C — Reproduction Checklist

The sequence that produced this build, in dependency order. Each step depends on the one before it, and performing them out of order accounts for most reported failures.

#	STEP	VERIFICATION
1	Create a dedicated Google Cloud project	Project appears with its own project number and ID.
2	Attach an active billing account	Service enablement stops being rejected — see Figure 1.
3	Enable the Smart Device Management API	Product page reads API Enabled — Figure 2.
4	Configure the consent audience as External	Audience page shows the External user type.
5	Promote publishing status to In production	Status reads In production — avoids the 7-day token expiry.
6	Create a Web application OAuth client	Client appears with an ID ending in the standard Google suffix.
7	Register the exact authorized redirect URI	URI matches byte for byte — Figure 5.
8	Enrol in Device Access and create a project	Project appears in the Device Access Console — Figure 3.
9	Bind the OAuth client ID to the Device Access project	Client ID visible in the project info panel.
10	Confirm the required SDM scope is present	Scope listed in the project info panel.
11	Run the Home Assistant setup flow and grant consent	User cap increments — Figure 4.
12	Establish the Pub/Sub topic and subscription	Pub/Sub shows as a configured integration — Figure 8.
13	Verify the partner connection was recorded	Connection listed against the home structure — Figure 6.
14	Confirm live API traffic and absence of errors	Request graph is non-zero, error reporting clean — Figure 7.
15	Validate device state and control end to end	Climate and sensor entities live and controllable — Figure 9.
16	Rotate the client secret with overlap	New secret enabled, superseded secret disabled — Figure 5.
17	Disable subscription expiry and plan for dormancy	Guards the 31-day and 6-month horizons — Section 7.3.

Appendix D — Figure Index

FIGURE	SUBJECT	VERIFIES
Figure 1	API enablement and billing prerequisite	Billing requirement enforced; SDM API enabled; consent audience baseline at 0 users
Figure 2	Smart Device Management API product page	Trust chain link 1 — the device interface is enabled on the project
Figure 3	Device Access project configuration	Trust chain link 3 — client binding, SDM scope, trait surface, tier limits
Figure 4	OAuth application state and audience	In-production status; user counter incremented by a completed grant
Figure 5	OAuth client, redirect URI and secrets	Trust chain link 2 — redirect binding and credential rotation with overlap
Figure 6	Nest partner connection	Trust chain link 4 — the homeowner's grant, from the user's side
Figure 7	Project dashboard request activity	Live API traffic at maintenance volume; no reported errors
Figure 8	Configured Cloud integrations	Pub/Sub and Logging provisioned; minimal footprint confirmed
Figure 9	Home Assistant dashboard	End-to-end validation — live, controllable, correctly rendered device state

ON EVIDENCE

Every figure is an unmodified capture from the live system apart from the permanent redaction of project numbers, project IDs, OAuth client IDs, client secrets, account identifiers and structure identifiers. No screenshot is a mock-up, a documentation image or a reconstruction.